

Embedded System

Introduction to Cortex-M4 Programming

Mostafiz Ahammed

Lecturer

*Department of IoT and Robotics Engineering
University of Frontier Technology, Bangladesh.*

Module Syllabus

- Program code
 - C language vs. assembly language
 - Program-generation flow
 - Cortex-M4 program image
- Program data
 - How is data stored in RAM?
 - Data types
 - Accessing data using C and assembly
- Mixed assembly and C programming
 - Calling a C function from assembly
 - Calling an assembly function from C
 - Embedded assembly

Program Code Overview

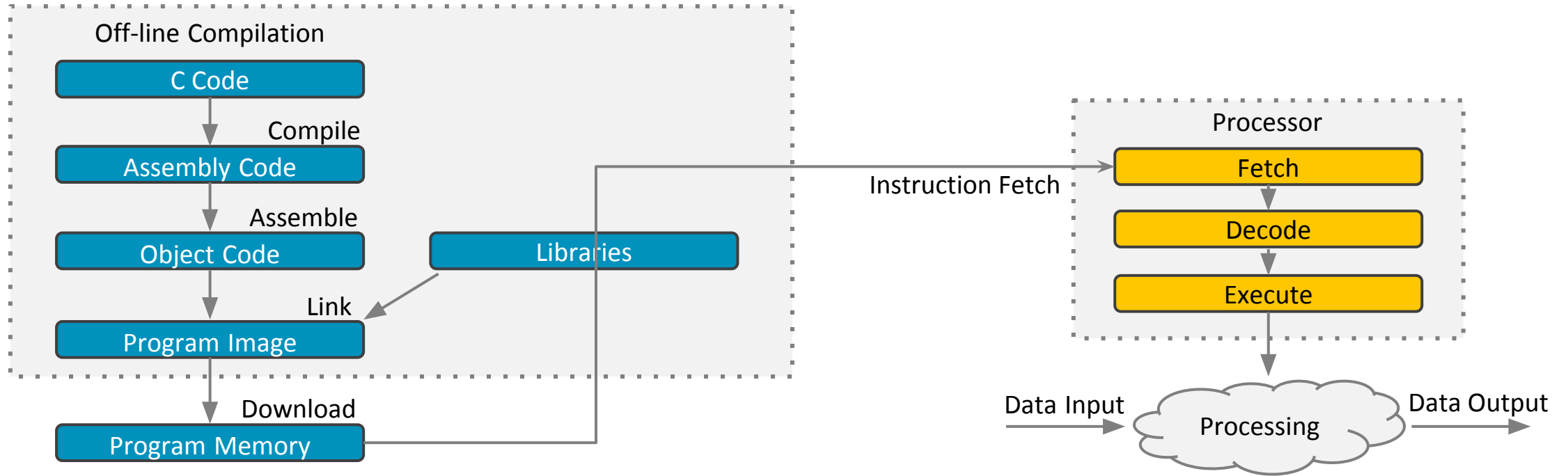
- All microcontrollers need a program code to perform an intended task.
- The program of embedded systems is usually developed at a relatively low level.
 - Less support from operating systems, as operating systems are partly unwanted overhead
 - Lack of standard programming interface
 - It is often hardware specific, depending on memory usage constraints, available instructions, etc.
- Programming language
 - Embedded systems are usually programmed using C (or C++) language and assembly language.
 - Arm-based tools can compile the C code or the assembly code to the executable file, which can be executed by Arm-based processors.
 - A completely integrated program code can also be referred as a program image or an executable file.

C Language vs. Assembly Language

Language	Advantages	Disadvantages
C	Easy to learn	Limited or no direct access to core registers and stack
	Portable	No direct control over instruction sequence generation
	Easy handling of complex data structures	No direct control over stack usage
Assembly	Allows direct control to each instruction step and all memory	Takes a longer time to learn
	Allows direct access to instructions that cannot be generated with C	Difficult to manage data structure
		Less portable

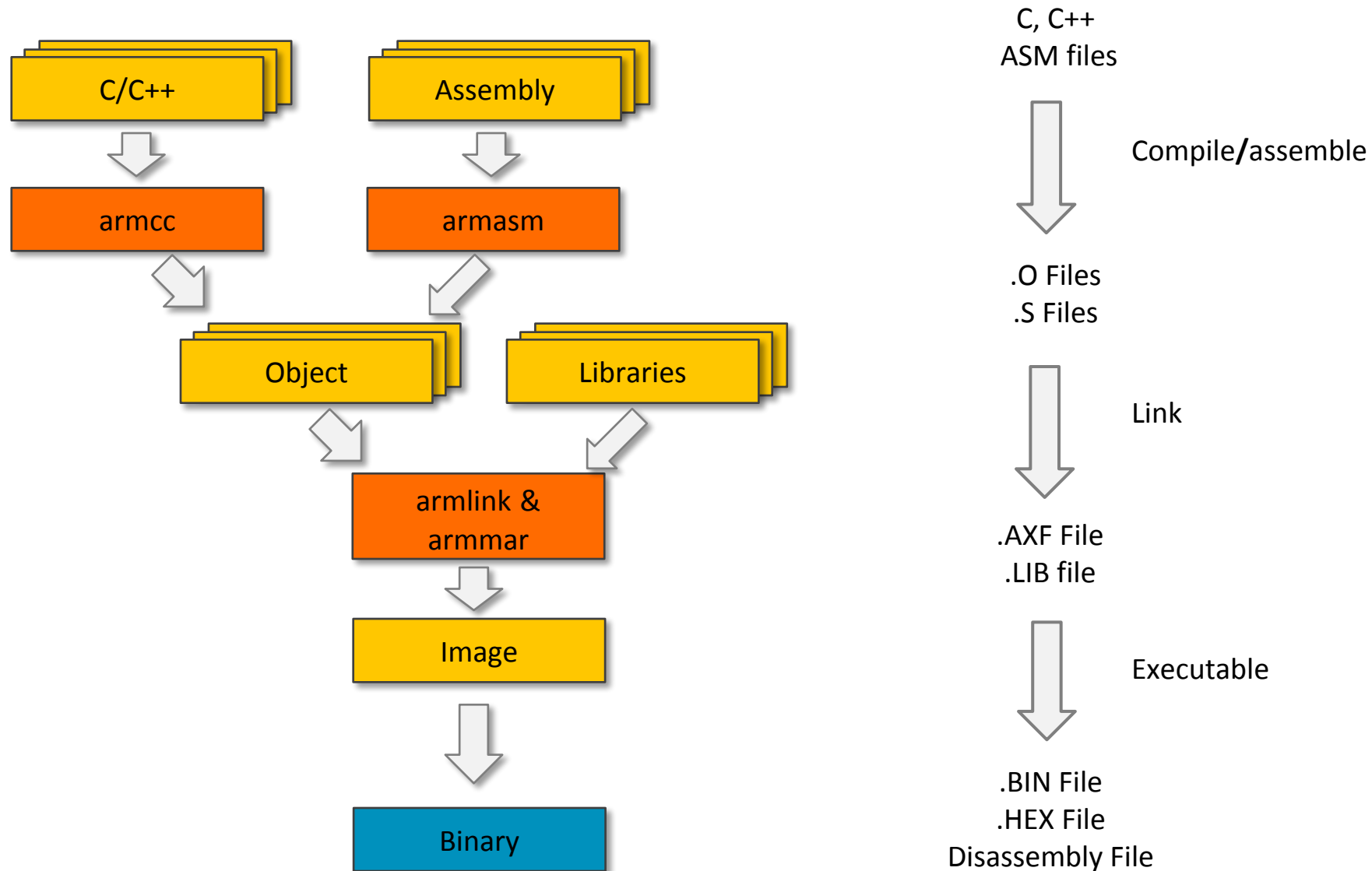
Typical Program-generation Flow

- The generation of program follows a typical development flow:
 - Compile -> Assemble -> Link -> Download
 - The generated executable file (or program image) is stored in the program memory (normally an on-chip flash memory), to be fetched by the processor



Typical program-generation flow

Compilation using Arm-based Tools

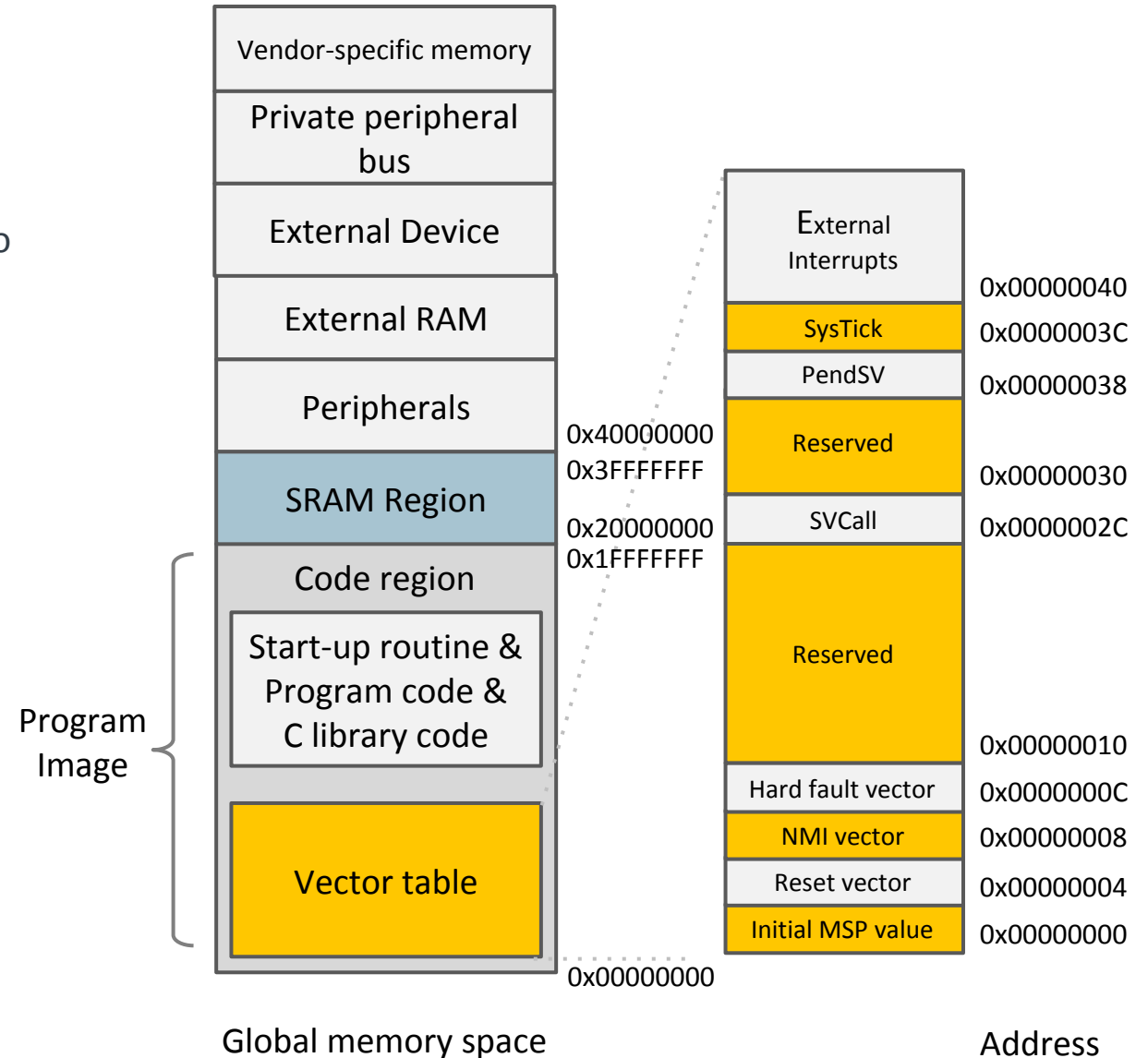


Compiler Stages

- Pre-processing
 - Replaces macros, defined by an initial hash-tag (#) in the code
 - Merges all subfiles (.c, .h) to one complete file
- Parser
 - Reads in C code
 - Checks for syntax errors
 - Forms intermediate code (tree representation)
- High-Level Optimizer: Modifies intermediate code (processor-independent)
- Code Generator
 - Creates assembly code step-by-step from each node of the intermediate code
 - Allocates variable uses to registers
- Low-Level Optimizer: Modifies assembly code (parts are processor-specific)
- Assembler: Creates object code (machine code)
- Linker/Loader: Creates executable image from object file

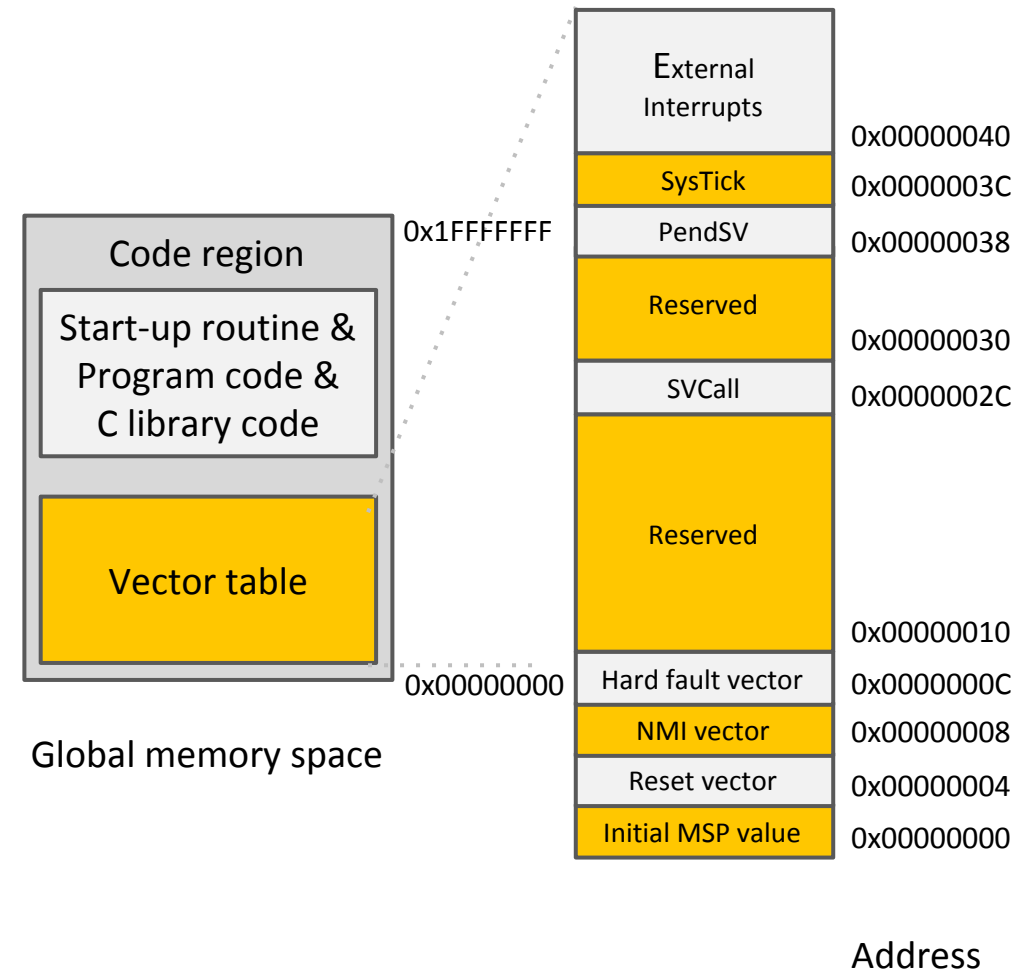
Cortex-M4 Program Image

- What is a program image?
 - The program image (sometimes also called the executable file) refers to a piece of fully integrated code that is ready to execute.
- In the Cortex-M4, the program image includes:
 - Vector table: includes the starting addresses of exceptions (vectors) and the value of the main stack point (MSP)
 - C start-up routine
 - Program code: application code and data
 - C library code: program codes for C library functions



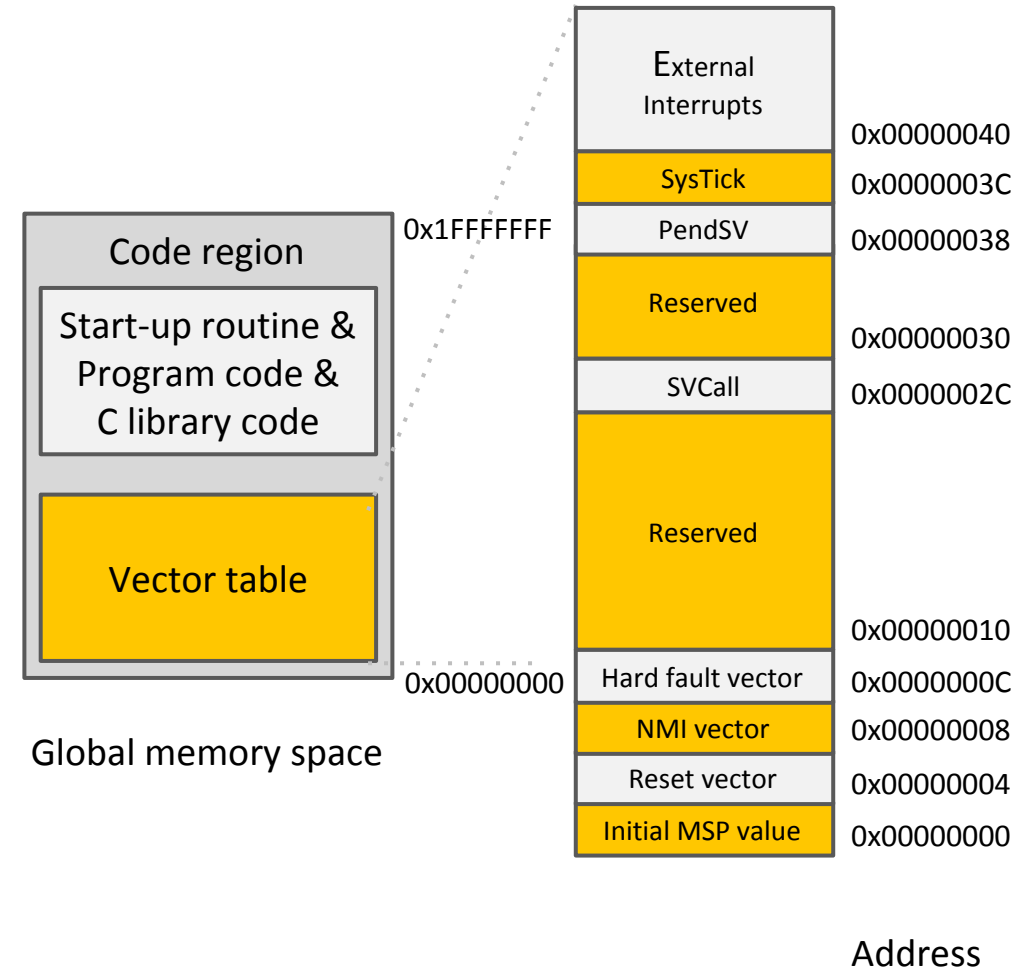
Cortex-M4 Program Image

- Vector table
 - Contains the starting addresses of exceptions (vectors) and the value of the main stack point (MSP)
- C Start-up code
 - Used to set up data memory and the initialization of values for global data variables
 - Is inserted by the compiler/linker automatically, labeled as ‘__main’ by the Arm compiler, or ‘__start’ by the GNU C compiler



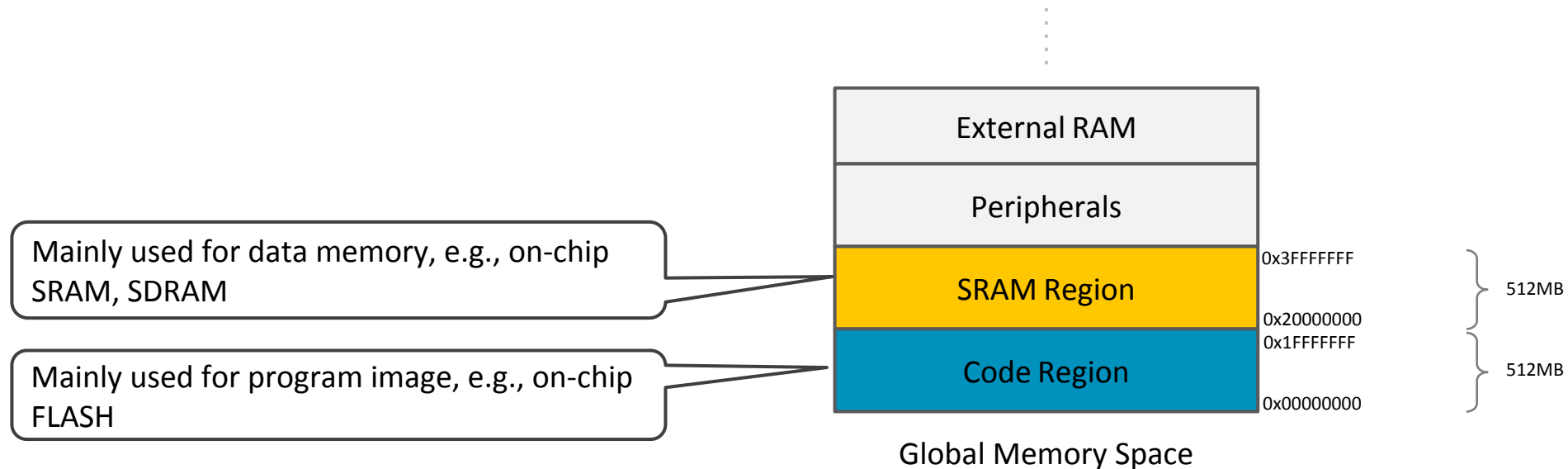
Cortex-M4 Program Image

- Program code
 - Program code refers to the instructions generated from the application program. Data types include:
 - Initial values of variables: the local variables that are initialized in functions or subroutines during program execution time
 - Constants: used in data values, address of peripherals, character strings, etc.
 - Sometimes stored together in data blocks called literal pools
 - Constant data such as lookup tables, graphics image data (e.g., bit map) can be merged into the program images
- C library code
 - Object codes inserted into the program image by linkers



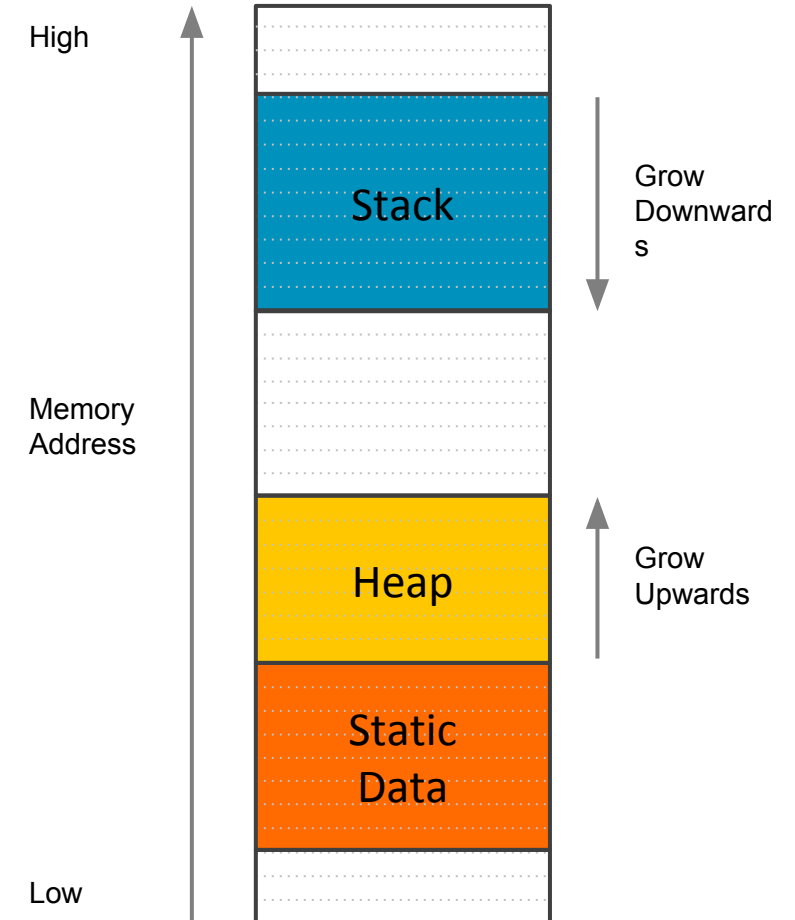
Program Image in Global Memory

- The program image is stored in the code region in global memory
 - Up to 512 MB memory space range from 0x00000000 to 0x1FFFFFFF
 - Usually implemented on non-volatile memory, such as on-chip FLASH memory
 - Normally separated from program data, which is allocated in the SRAM region (or data region)



How is Data Stored in RAM?

- Typically, the data can be divided into three sections: static data, stack, and heap
 - Static data: contains global variables and static variables
 - Stack: contains the temporary data for local variables, parameter passing in function calls, registers saving during exceptions, etc.
 - Heap: contains the pieces of memory spaces that are dynamically reserved by function calls, such as 'alloc()' and 'malloc ()'



Defining the Stack and Heap

- The stack and heap can be defined in either C language (with linker file) or assembly language, for example, in C:

```
/* Set stack and heap parameters */

#define STACK_BASE    0x10020000          // stack start address
#define STACK_SIZE    0x5000             // length of the stack
#define HEAP_BASE     0x10001000         // heap starts address
#define HEAP_SIZE     0x10000 - 0x6000 // heap length

/* inker generated stack base addresses */

extern unsigned int Image$$Arm_LIB_STACK$$ZI$$Limit
extern unsigned int Image$$Arm_LIB_STACKHEAP$$ZI$$Limit

...
```

Defining the Stack and Heap

- The stack and heap can be defined in assembly language as follows:

```
Stack_Size      EQU    0x00000400      ; 256KB of STACK

                AREA    STACK, NOINIT, READWRITE, ALIGN=4
Stack_Mem
__initial_sp    SPACE    Stack_Size

Heap_Size       EQU    0x00000400      ; 1MB of HEAP

                AREA    HEAP, NOINIT, READWRITE, ALIGN=4
__heap_base
Heap_Mem        SPACE    Heap_Size
__heap_limit
```

Data Types

- A number of standard data types are supported by the C language
- However, their implementation depends on the processor architecture and C compiler
- In Arm programming, the data size is referred to as byte, half word, word, and double word:
 - Byte: 8-bit
 - Half word: 16-bit
 - Word: 32-bit
 - Double word: 64-bit
- The table in the next slide shows the implementation of different data types

Data Types

Data type	Size	Signed Range	Unsigned Range
char, int8_t, uint8_t	Byte	-128 to 127	0 to 255
short, int16_t, uint16_t	Half word	-32768 to 32767	0 to 65535
int, int32_t, uint32_t, long	Word	-2147483648 to 2147483647	0 to 4294967295
long long, int64_t, uint64_t	Double word	-2^{63} to $2^{63}-1$	0 to $2^{64}-1$
float	Word	$-3.4028234 \times 10^{38}$ to 3.4028234×10^{38}	
double, long double	Double word	$-1.7976931348623157 \times 10^{308}$ to $1.7976931348623157 \times 10^{308}$	
pointers	Word	0x00 to 0xFFFFFFFF	
enum	Byte/ half word/ word	Smallest possible data type	
bool (C++), _bool(C)	Byte	True or false	
wchar_t	Half word	0 to 65535	

Data Type and Class Qualifiers

Const

- Never written by program, can be put in ROM to save RAM

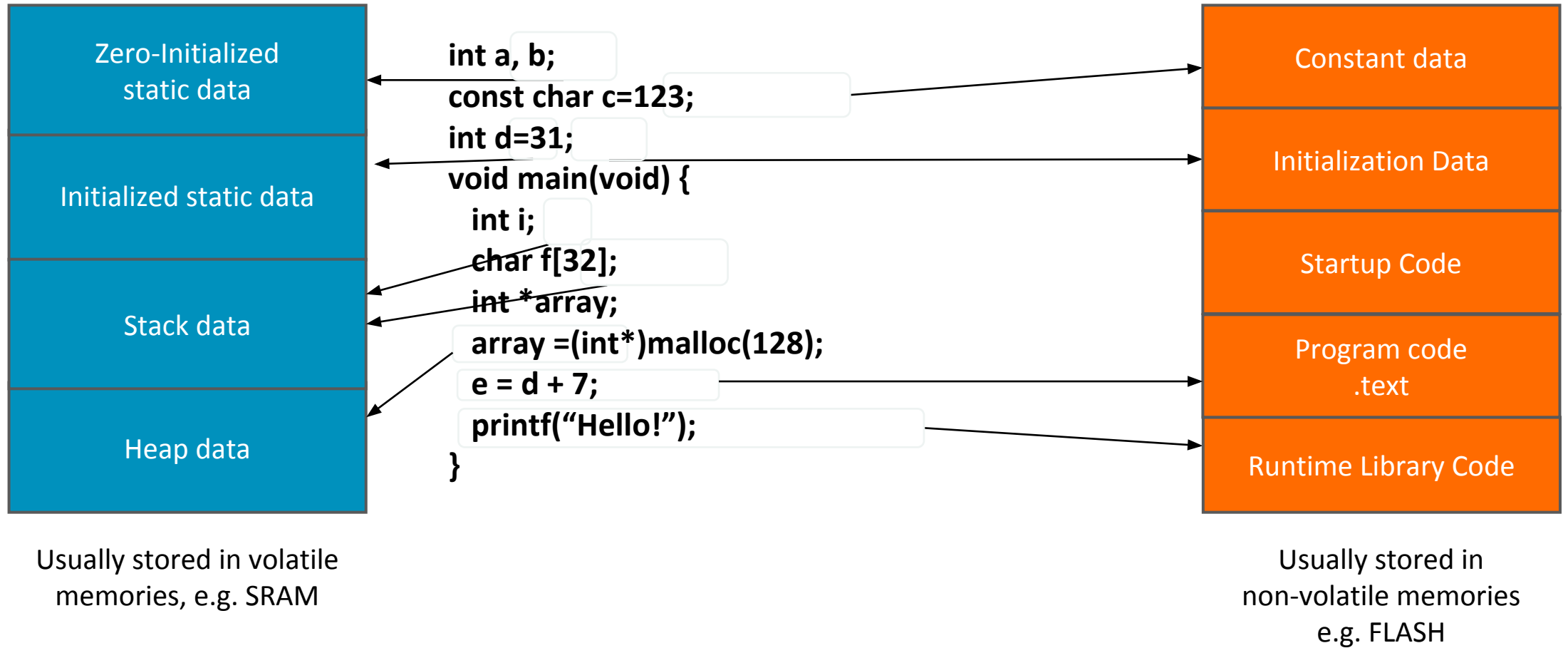
Volatile

- Can be changed outside of normal program flow: ISR, hardware register
- Compiler must be careful with optimizations

Static

- Declared within function, retains value between function invocations
- Scope is limited to function

Example of Data Storage



Defining the Interrupt Vector in C

- The interrupt vector can be defined in either C language or assembly language, for example, in C:

```
typedef void(* const ExecFuncPtr)(void) __irq;

#pragma arm section rodata="exceptions_area"

ExecFuncPtr exception_table[] = {
    (ExecFuncPtr)&Image$$Arm_LIB_STACK$$ZI$$Limit, /* Initial SP */
    (ExecFuncPtr)__main,                          /* Initial PC */
    NMIAException,
    HardFaultException,
    MemManageException,
    BusFaultException,
    UsageFaultException,
    0, 0, 0, 0, /* Reserved */
    SVCHandler,
    DebugMonitor,
    0, /* Reserved */
    PendSVC,
    SysTickHandler
    /* Configurable interrupts start here... */
};
#pragma arm section
```

Accessing Peripherals in C

- Define base addresses for peripherals:

```
#define FLASH_BASE    *((volatile unsigned long *) (0x00000000UL))
#define RAM_BASE      *((volatile unsigned long *) (0x10000000UL))
#define GPIO_BASE     *((volatile unsigned long *) (0x2009C000UL))
#define APB0_BASE     *((volatile unsigned long *) (0x40000000UL))
#define AHB_BASE      *((volatile unsigned long *) (0x50000000UL))
#define CM3_BASE      *((volatile unsigned long *) (0xE0000000UL))
```

- Write a value to a peripheral register:

```
//store a value to the memory
RAM_BASE = 0x3FFFF
```

- Read a value from a peripheral register:

```
//read a value from the memory
i= RAM_BASE;
```

Using Bit-band Operation in Assembly

- For example, in order to set bit[3] in word data in address 0x20000000
 - Read-Modify-Write operation
 - Read the real data address (0x20000000)
 - Modify the desired bit (retain other bits unchanged)
 - Write the modified data back
 - Bit-band operation
 - Directly set the bit by writing '1' to address 0x2200000C, which is the alias address of fourth bit of the 32-bit data at 0x20000000

;Read-Modify-Write Operation

```
LDR    R1, =0x20000000 ;Setup address
LDR    R0, [R1]         ;Read
ORR.W  R0, #0x8         ;Modify bit
STR    R0, [R1]         ;Write back
```

;Bit-band Operation

```
LDR    R1, =0x2200000C ;Setup address
MOV    R0, #1          ;Load data
STR    R0, [R1]        ;Write
```

Use Bit-band Operation in C

- The bit-band operation is not natively supported in most C compilers
- For ease of use of the bit-band feature, the address and the bit-band address can be separately declared, for example:

```
#define RAM_Data1      *((volatile unsigned long *) (0x20000000))
#define RAM_Data1_Bit0 *((volatile unsigned long *) (0x22000000))
#define RAM_Data1_Bit1 *((volatile unsigned long *) (0x22000004))

RAM_Data1 = RAM_Data1 | 0x01      ;Setup bit0 without using bit-band
RAM_Data1_Bit0 = 0x00            ;clear bit0 using bit-band
RAM_Data1_Bit1 = 0x01            ;Setup bit1 using bit-band
```

- Alternatively, C macros can be used to easily access bit-band alias, for example:

```
#define RAM_Data1      *((volatile unsigned long *) (0x20000000))
#define bit-band(data_addr, bit) ((data_addr & 0x0F000000) + 0x2000000 +  
((data_addr & 0xFFFFF) << 5) + (bit << 2))
#define Data_Addr (data_addr)      *((volatile unsigned long *) (data_addr))

Data_Addr(bit-band(RAM_Data1, 0)) = 0x01      ;set bit0
```

Calling a C Function from Assembly

- When a C function is called from an assembly file, be aware of the following areas:
 - Register R0, R1, R2, R3, R12, and LR could be changed; hence, it is better to save them to the stack.
 - The value of SP should be aligned to a double-word address boundary.
 - Input parameters must be stored in the correct registers, for example, registers R0 to R3 can be used for passing four parameters.
 - The return value is usually stored in R0.

Calling a C Function from Assembly

- For example, writing an adding function in C:

```
int my_add(int x1, int x2, int x3, int x4) {  
    return (x1+x2+x3+x4);  
}
```

- Call the C function from the assembly code, for example:

```
MOVS    R0, #0X1        ; parameter x1  
MOVS    R1, #0X3        ; parameter x2  
MOVS    R2, #0X5        ; parameter x3  
MOVS    R3, #0X7        ; parameter x4  
IMPORT  my_add  
BL      my_add           ; call "my_add" function in C
```

Calling an Assembly Function from C

- When calling an assembly function from C code, you should be aware of the following:
 - If registers R4 to R11 need to be changed, they must be stacked and restored in the assembly function
 - If another function is called inside the assembly function, the LR needs to be saved on the stack and used for return
 - The function return value is normally stored in R0

Calling an Assembly Function from C

- Writing a function in assembly, for example:

```
add_asm    EXPORT add_asm
           FUNCTION
           ADDS    R0, R0, R1
           ADDS    R0, R0, R2
           ADDS    R0, R0, R3
           BX      LR      ; result is returned in R0
           ENDFUNC
```

- Calling an assembly function in C, for example:

```
external int add_asm( int k1, int k2, int k3, int k4);
void main {
    int x;
    x = add_asm (11,22,33,44);    // call assembly function
    ...
}
```

Embedded Assembly

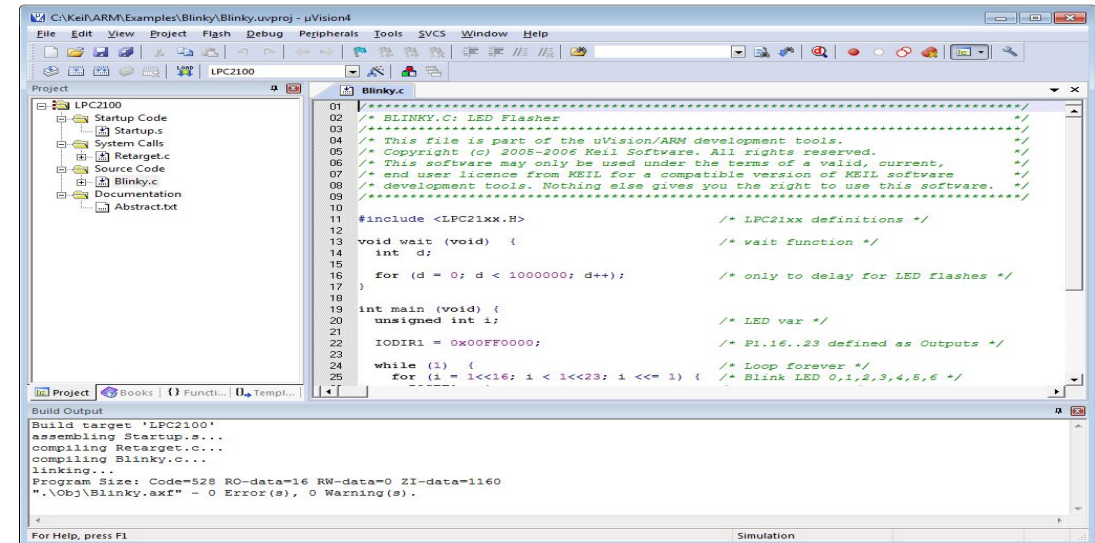
- The embedded assembler allows developers to write assembly functions inside C files, for example, in C:

```
_asm int add_asm( int k1, int k2, int k3, int k4) {  
    ADDS    R0, R0, R1  
    ADDS    R0, R0, R2  
    ADDS    R0, R0, R3  
    BX      LR  
}  
  
void main {  
    int x;  
    x = add_asm (11,22,33,44);    // call assembly function  
    ...  
}
```

Arm Keil[®] MDK

- Keil μ Vision has an Integrated Development Environment (IDE), which allows the user to build a project easily and quickly. The IDE includes:
 - Project management
 - Build facilities
 - Source code editing
 - Program debugging
 - Complete simulation
- A series of Arm-based tools are integrated in Keil μ Vision, including:
 - Compiler
 - Assembler
 - Linker
 - Format converter
 - Libraries

arm KEIL



Resources

- Cortex-M4 Technical Reference Manual: <https://developer.arm.com/docs/100166/0001>
- Cortex-M4 Devices Generic User Guide: <https://developer.arm.com/docs/dui0553/b>